
The development X
Lean, and the paper's notation
The bundle b = extract_X(rs)
bundle.json, for the five requests rs of Figure 2
The paper d = splice_b(d_0)
the TeX of Figure 2, its blocks from b
Stlc.lean
inductive Step : Tm → Tm → Prop
/-- β-reduction -/
| beta {t e v} (hv : IsValue
v) : Step (.app (.lam t e)
v) (subst e v)
...
theorem values_final {v e :
Tm} (hv : IsValue v) : ¬
Step v e := by
StlcTeX.lean
latex_notation Stlc.Step e e'
=> r"#e \longrightarrow #e'"
prec := 10
latex_notation Stlc.Tm.lam t e
=> r"\lambda{:}#t.\, #e"
prec := 0
latex_notation Stlc.Tm.app e₁
e₂ => r"#e₁\; #e₂" prec :=
100
latex_notation Stlc.Tm =>
r"\mathit{Term}"
latex_metavar_convention
Stlc.IsValue v => r"V"
latex_category Stlc.Tm => "M"
latex_entry "Values are final"
=> Stlc.values_final
...
extract_X
extractor
{ "entries": {
"Values are final": {
"decl": "Stlc.values_final",
"kind": "theorem",
"axioms": [],
"sorry": false,
"stmtHash":
"4ad202811bf6...",
"views": {
"prose": "\\LeanProse{Let
$V \\in \\mathit{Val}$
and $M \\in
\\mathit{Term}$. Then
$\\neg (V
\\longrightarrow M)$.}",
"lean": "\\LeanCode{%\n\\Lea
nKeyword{theorem}
\\LeanConst{values_final
} \\{\\LeanBound{v}
\\LeanBound{e} :
\\LeanConst{Tm}\\}
(\\LeanBound{hv} :
\\LeanConst{IsValue}
\\LeanBound{v}) :
¬\\LeanConst{Step}
\\LeanBound{v}
\\LeanBound{e}}"
}
}
... and 4 more entries
} }
splice_b
splicer
\begin{leantheorem}[sum=8jx5_cP
cFh]{Values are
final}{Values are final}
\label{thm:Values-are-final}
\LeanProse{Let $V \in
\mathit{Val}$ and $M \in
\mathit{Term}$. Then $\neg
(V \longrightarrow M)$.}
\end{leantheorem}
\begin{leanrules}[sum=5cW2xFjnl
Q]{Stlc.Step}{Reduction}
\begin{mathpar}
\inferrule[(Beta)]
{ }
{ (\lambda{:}A.\, M)\; V
\longrightarrow M[V] }
...
\begin{leansource}[sum=zIy9s_XP
en]{Stlc.values_final}{The
source}
\LeanCode{%
\LeanKeyword{theorem}
\LeanConst{values_final}
\{\LeanBound{v}
\LeanBound{e} :
\LeanConst{Tm}\}
(\LeanBound{hv} :
\LeanConst{IsValue}
\LeanBound{v}) :
¬\LeanConst{Step}
\LeanBound{v} \LeanBound{e}}
\end{leansource}
the statement's
each rendering
world(X): read back by the validator's parser, each formula and its one reading
V ↦ V
\mathit{Val} ↦ Stlc.IsValue
M ↦ M
\mathit{Term} ↦ Stlc.Tm
\neg (V \longrightarrow M) ↦ Not[Stlc.Step[V; M]]
(\lambda{:}A.\, M)\; V \longrightarrow M[V]
↦ Stlc.Step[Stlc.Tm.app[Stlc.Tm.lam[A; M]; V]; Stlc.subst[M; V]]
world(X) ⊨ b (Theorem 1.1), and
check_b(d) = [], so world(X) ⊨ d
and pollack_X(d) (Theorem 1.3):
every formula has one reading,
and it is the one it was
rendered from